

ポートフォリオ

ASOポップカルチャー専門学校
ゲーム・CG専攻科 ゲーム専攻

東島 裕大

2026卒

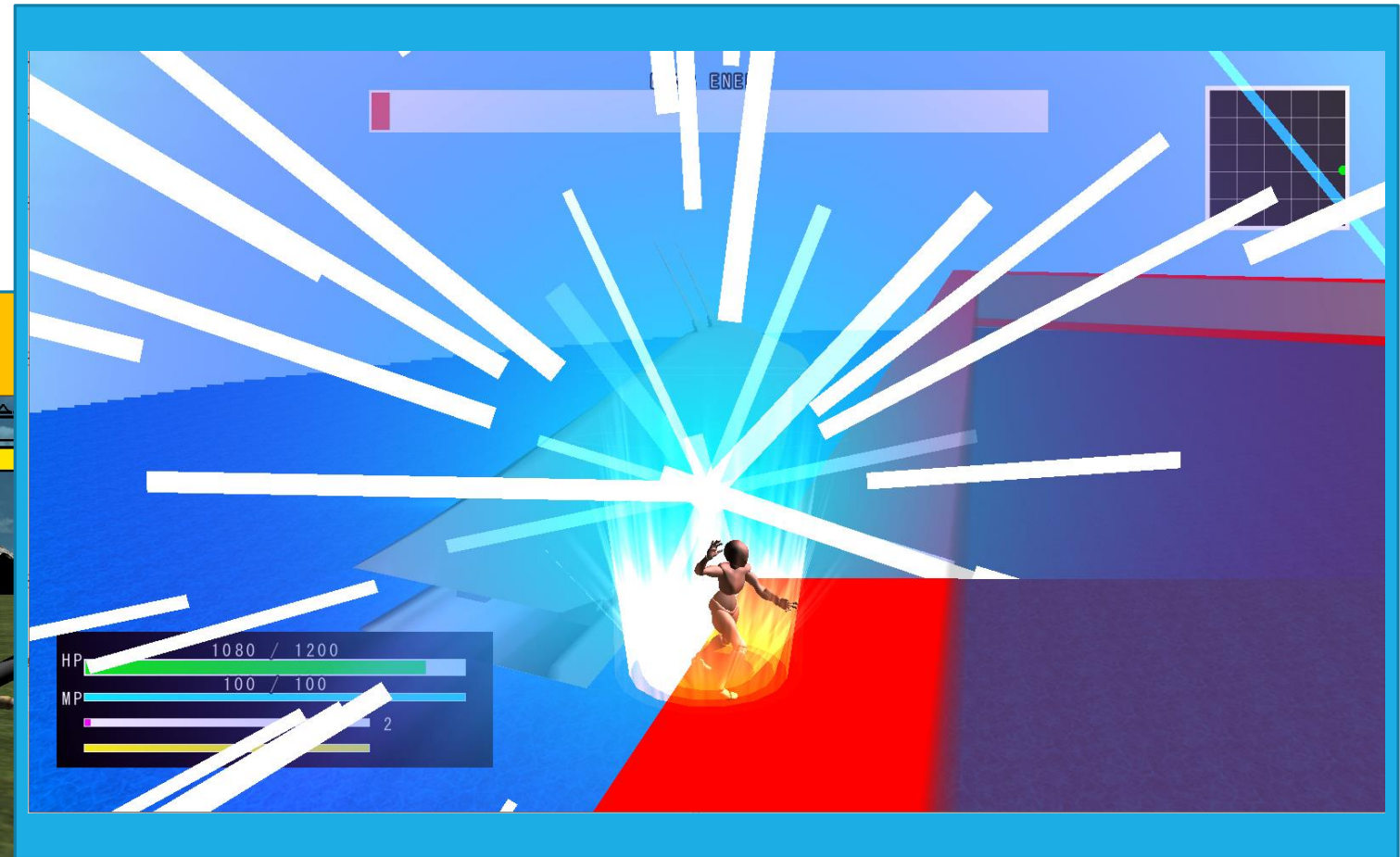
目次

2 ——— 3年次制作作品

10 ——— 2年次制作作品

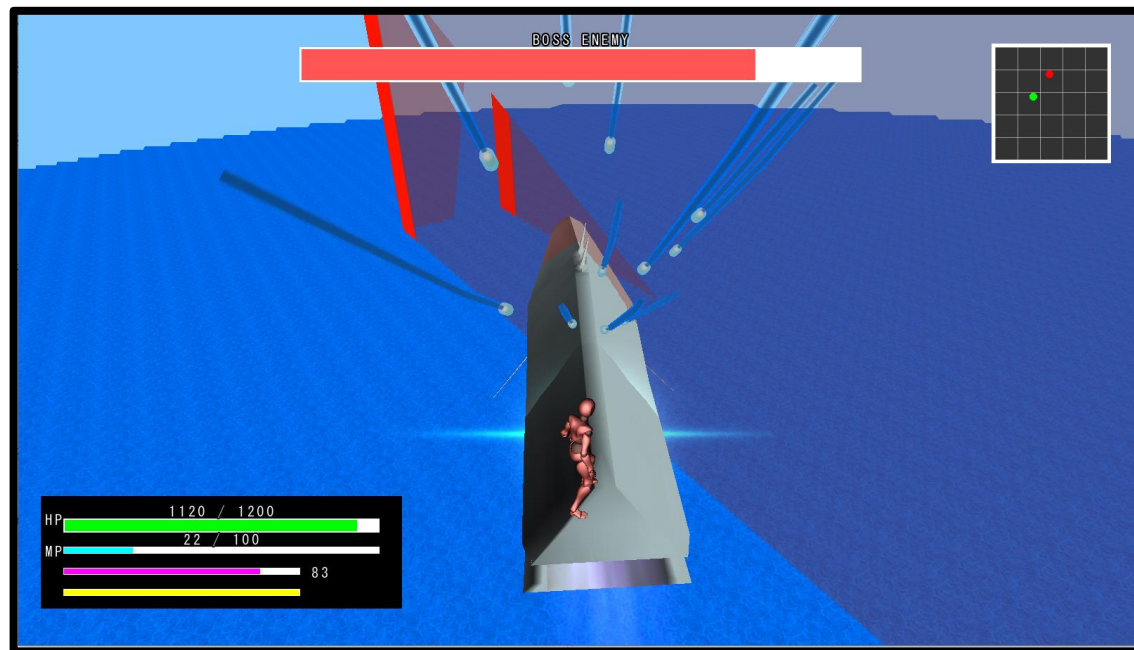
13 ——— 授業制作作品

3年次



Sky Shooter

学内コンテスト用の作品で入賞した作品になります。
前作品で肥大化していた入力制御機能の分割から始まり、衝突や入力の通知を行うObserverパターンの実装、FactoryMethodパターンを用いたオブジェクトの生成、リソースの非同期読込の実装、マルチスレッドを用いた衝突判定の実装などを主軸に行いました。
ゲームのボリュームがまだまだ薄いため、ゲームの要素を増やす更新を行っていきたいと考えています。



開発期間 : 9月 ~ 1月

使用言語 : C++20、HLSL

使用ライブラリ等 : DXライブラリ, nlohmannJSON, Effekseer, GitHub

制作人数 : 1人

Observer(オブザーバー)

```
std::function<void(void)> update;

for (const auto& mes : observer_.ReceiveMessageList())
{
    if (!messageUpdate_.contains(mes))
    {
        //通知が登録されていない
        continue;
    }

    update = messageUpdate_.at(mes);

    if (!update)
    {
        //関数オブジェクトがリンク切れ
        continue;
    }

    update();
}
```

```
void PlayerUnit01::IsHit_EnemyMissile(void)
{
    if (IsAbleShield() && !IsShieldCounter() && !IsInvisible())
    {
        //シールド中かつ無敵時間ではない
        CounterCount();
        return;
    }

    if (!IsInvisible())
    {
        //被弾
        Damage(ENEMY_SHOT_MISSILE_DAMAGE);
        invFrameCnt_ = deltaTime_ * INV_FRAME_MAX;

        UCHeal(UC_HEAL_HIT_ENEMYATTACK);

        uiDataMng_.SetUIData(SRC_UIData::PLAYER_HP, hp_);
    }
}
```

前回の作品での欠点の解決を目指し、オブザーバーの実装をすることで、解決を図ることが出来るのではないかと考え、実装しました。

1f前に出された通知を受け取り、フレームの始めに消化する仕様にしています。

衝突判定が1つのクラスに集約され、どのオブジェクトと衝突しており、どの処理が行われているかがかなり分かりやすくなったと感じました。

更に、副作用として、マルチスレッドを利用した衝突判定の実装を行う際の拡張もスムーズに行うことが出来ました。

マルチスレッドの実装時に行ったこと

```
lock_guard<mutex> lock(col3Dmutex_);

auto& charactors = charaMng_.GetCharactors();
if (charactors.empty())
{
    //キャラクターの配列が存在しない
    return;
}

auto& shots = shotMng_.GetShots();
if (shots.empty())
{
    //弾の配列が存在しない
    return;
}

//キャラと弾
for (const weak_ptr<CharactorBase> chara : charactors)
{
    if (chara.expired())
    {
        //キャラの実体が存在しない
        continue;
    }
}
```

Qiitaに
マルチスレッドに関連した記事を投稿することで、
有識者に意見を求め、
自身の認識が大きく間違っていないかの確認を
行いました。

std::mutexとstd::lock_guardを使用した
排他制御を行い、
スレッドセーフを実現しています。

Qiita :
https://qiita.com/Qiita_No_Namaco





学内コンテスト用の作品で、1位入賞した作品になります。

初めての長期間でのチーム制作となり、苦労する点が多かったです。

キーボード、コントローラー、マウスに対応した入力制御の制作、カメラの制御、リソースの管理クラス、シェーダーを使用しやすくするための機能、オブジェクトの衝突判定などを担当しました。



開発期間 : 4月～8月

使用言語: C++20,HLSL

使用ライブラリ等 : DXライブラリ,nlohmannJSON, Unity(ステージモデル制作), Effekseer,GitHub

制作人数 : 5人

入力制御の説明

```
//通常攻撃
inputActionMaps_[ACTION::ATTACK] =
{
    {InputType::GAMEPAD, (int)JOYPAD_BTN::TOP},
    {InputType::MOUSE, MOUSE_INPUT_LEFT},
};

//ポーズ
inputActionMaps_[ACTION::PAUSE] =
{
    {InputType::KEYBOARD, KEY_INPUT_P},
    {InputType::GAMEPAD, (int)JOYPAD_BTN::START},
};
```

```
//それぞれのアクション名に割り当てている全ての入力チェック
for (const auto& mapInfo : inputActionMaps_)
{
    bool isPressed = false;

    for (const auto& inputInfo : mapInfo.second)
    {
        isPressed = GetHitKey(inputInfo, keyState, padState, mouseState);
        if (isPressed)
        {
            currentInputType_ = inputInfo.type;
            //入力していたらループ終了
            break;
        }
    }

    //入力情報を更新
    currentInput_[mapInfo.first] = isPressed;
}
```

キーマウ操作、コントローラー操作に対応した入力制御を作る必要があり、アクションゲームということもあり、対応するアクションも多かった

アクション名をキー値として、連想配列にアクション名に対応した入力を登録し、割り当てられた入力情報から入力検知を行っています。

欠点として、InputManagerクラスで登録、更新、入力判定を全てこなしているため、かなり肥大化してしまっていることなどがあります。

衝突判定に関して

```
// 頭
scnMng_.GetColManager().AddCollider(
    OBJECT_TYPE::ENEMY_VOL, COL_TYPE::SPHERE,
    transform_,
    true, -1, mouthFrame_, ENEMY_SUBTYPE::HEAD, HEAD_COL_RADIUS);

// ステージとの判定
scnMng_.GetColManager().AddCollider(
    OBJECT_TYPE::ENEMY_SM_SPHERE, COL_TYPE::SPHERE,
    transform_,
    true, -1, mouthFrame_, ENEMY_SUBTYPE::NONE, STAGE_COL_RADIUS);

// 全体
scnMng_.GetColManager().AddCollider(
    OBJECT_TYPE::ENEMY_VOL_MODEL, COL_TYPE::MODEL,
    transform_, true, -1, -1);
```

```
// コリジョンセット
colliderController_>SetCollision(OBJECT_TYPE::ENEMY_VOL);
colliderController_>SetCollisionSub(ENEMY_SUBTYPE::HEAD);

const auto& hits = colliderController_>OnCollision(OBJECT_TYPE::PLAYER);
const auto& hitpoly = colliderController_>OnCollisionResultPoly(OBJECT_TYPE::PLAYER);
```

```
if (hits && !(player_.lock()->GetInvincibility()))
{
    player_.lock()->IsImpact(Player::PATTERN_NUM::TWO);
    if (player_.lock()->GetSTATE() != Player::STATE::BLOCK)
    {
        player_.lock()->SetHitDir(hitDir, BLOWING_AWAY);
        player_.lock()->Damage(ROCK_DAMAGE);
        player_.lock()->SetInvincibility(true);
    }
}
```

衝突判定に関する全体的な労力を削減したかった

ColliderManagerに衝突判定情報を登録することで、
衝突する対象の名前と
追従しているフレーム名を知っていれば
衝突判定の実装が可能になっています。

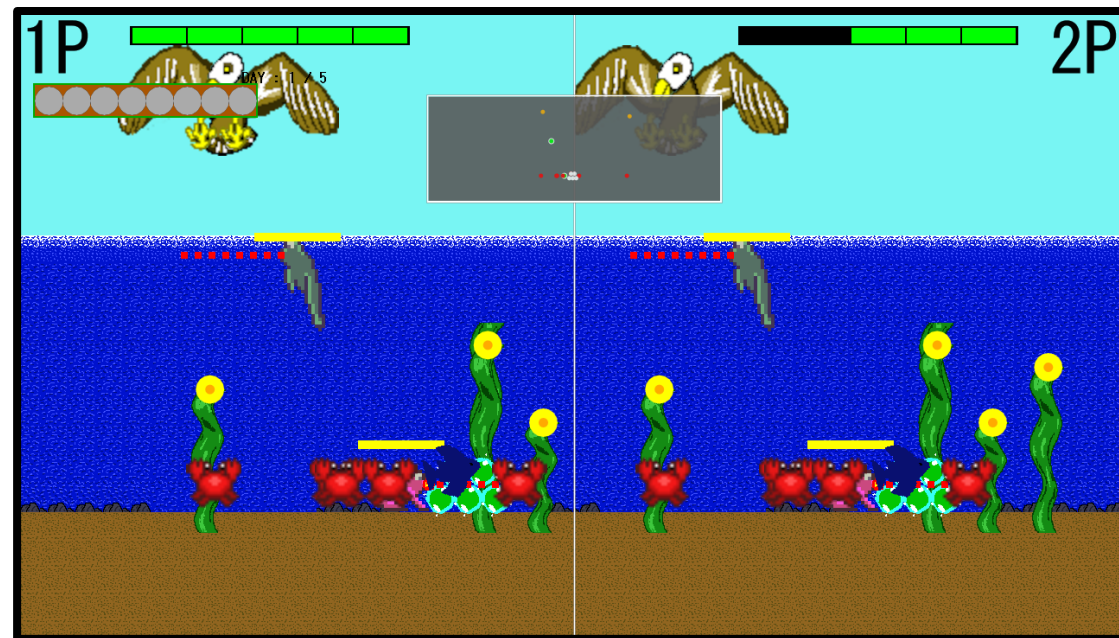
欠点として、
衝突時の処理が散らばってしまい、
相手側が衝突時の処理をどこで行っているのかが
分かりづらいこと、
衝突時、相手に命令を出す仕様のため、
結局相手の情報が必要になってしまうことがあり、
利点を捨ててしまっていることなどがあります。
この欠点を解決するために
次作品でObserverパターンを用いて実装を行いました。

カエル帰るそして卵を守る

春季休暇中の部活動の活動内容として制作し、部内のコンテストで3位に入賞しました。

初のチーム制作となった他、後輩に指導しつつ、1ヶ月半程度という期間で制作を進めることとなり、苦労する点は多かったです。

カメラやオブジェクトの衝突判定、敵、水草、岩、餌の制御、HPなどのUI、ピクセルシェーダを用いたポストエフェクト、ミニマップの作成などを担当しました。



開発期間 : 2月～3月

使用言語 : C++20,HLSL

使用ライブラリ等 : DXライブラリ,GitHub

制作人数 : 4人

2年次



LASER

学内コンテスト用の作品となります。

モデルやゲームシステムを他作品から流用し、HLSLを用いたシェーダに関する学習に時間をかけました。

リムライトやスキニングメッシュ対応の頂点シェーダに力を入れ、

その他にも、関数ポインタやラムダ式、スマートポインタの学習、実装も行いました。



開発期間 : 9月～1月

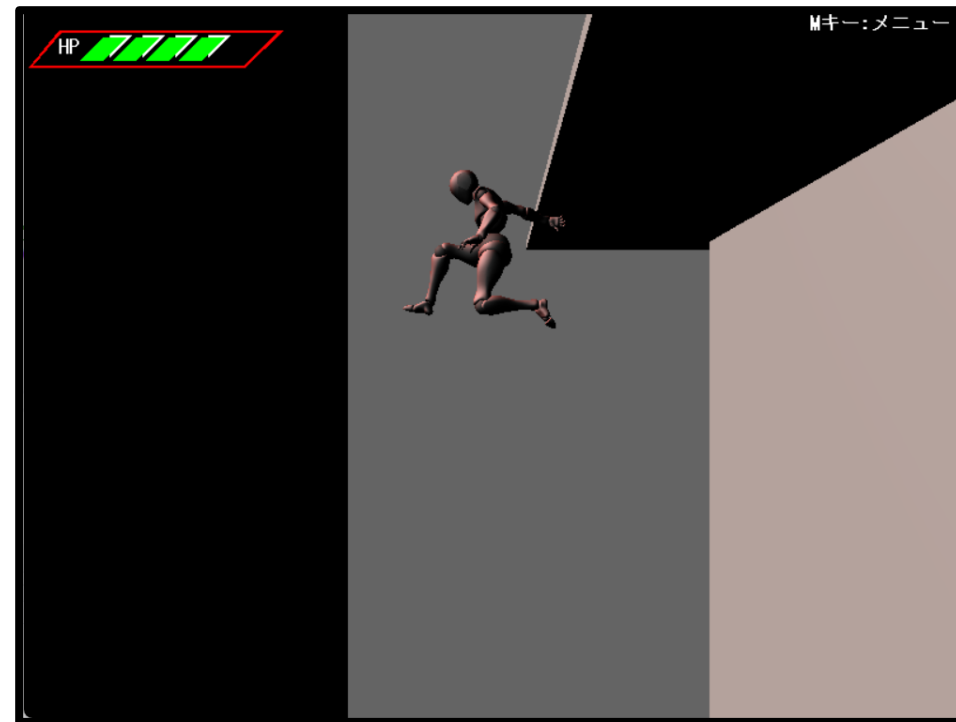
開発環境 : C++20, HLSL

使用ライブラリ等 : DXライブラリ, Effekseer, GitHub

制作人数 : 1人

RUNNER

初めて学内コンテスト提出した作品となります。
3Dで制作を行い、キャラの状態の制御、アニメーションブレンドに力を入れて制作を行いました。
プレイヤーの状態遷移や衝突判定にかなりの時間を取られてしまい、
最低限のゲームループが実装されている程度の作品になってしまいました。



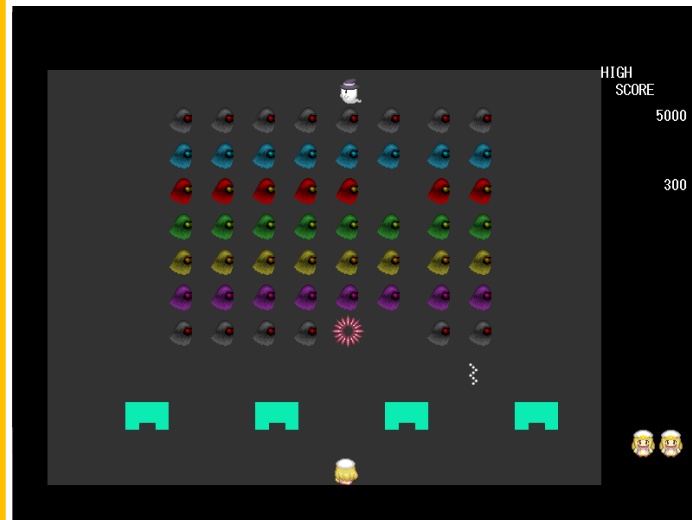
開発期間 : 4月～8月
開発環境 : C++14
使用ライブラリ等 : DXライブラリ
Blender(ステージ制作)
制作人数 : 1人

授業作品

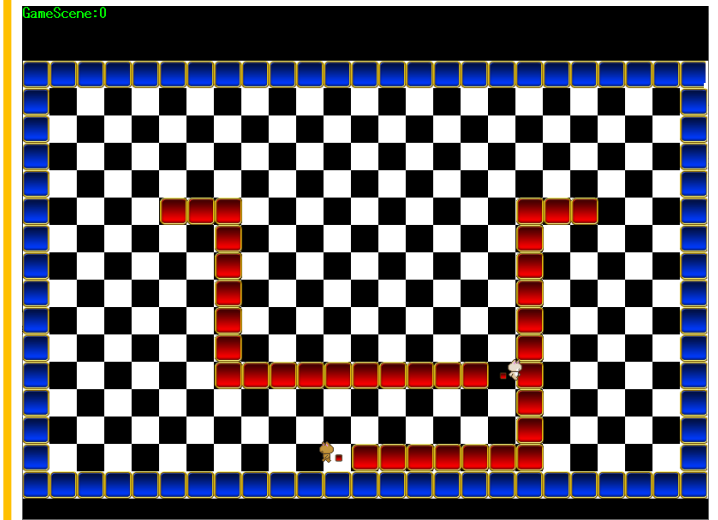




シューティングゲーム
1年前期
初のDXライブラリを用いた
ゲームの制作
C言語を使用して
一通りのゲームループを実装



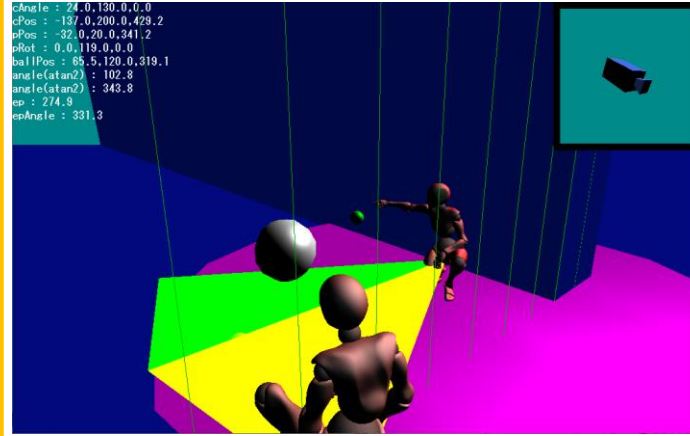
インベーダーゲーム
1年前期
配列を利用したゲームの制作



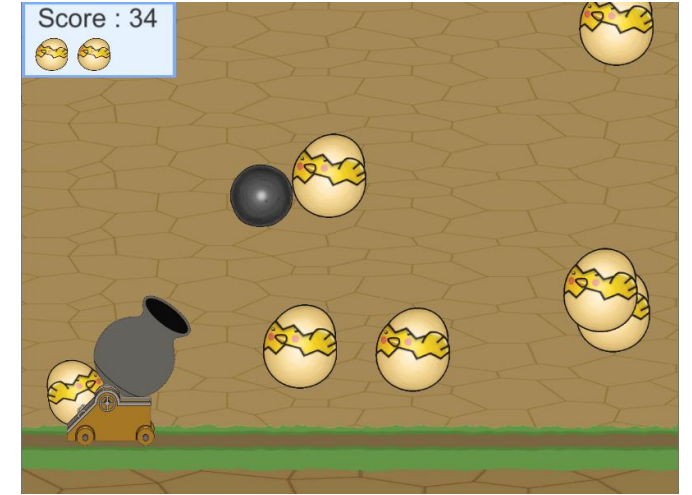
スネークゲーム
1年前期
初のC++言語を使用し
てのゲーム制作
クラスの実装方法を学ぶ



3Dキャノン
2年前期
初の3Dゲームの制作
3Dにおける行列回転などを行う



3Dワールド
2年前期
クォータニオンを利用した回転や
個人的にアニメーションブレンド
等を行う



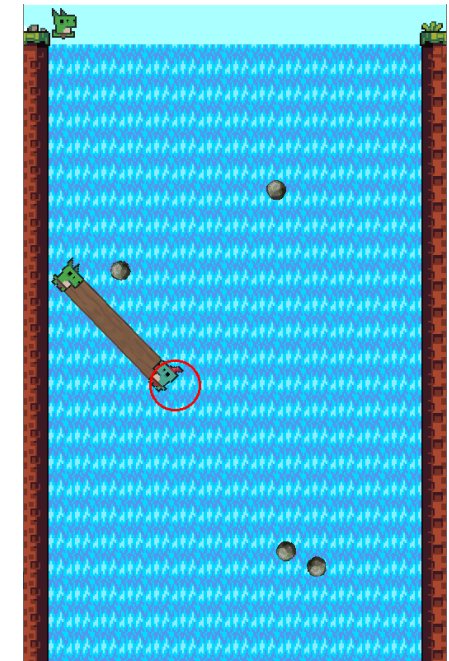
2Dシューティングゲーム
2年前期
Unityを用いた2Dゲーム



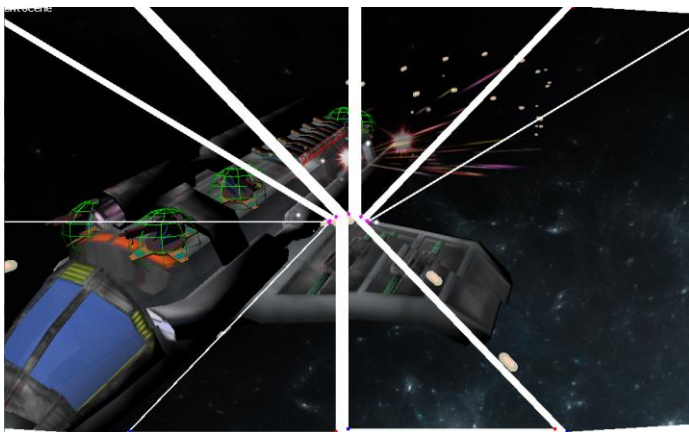
ピクセルシェーダ
2年前期
初のピクセルシェーダを用いた
ポストエフェクトの実装



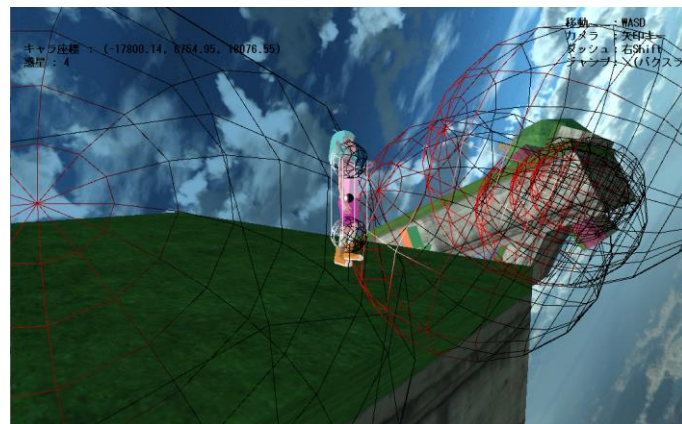
シューティングゲーム
2年前期
ベクトルを用いて
弾幕シューティングを作成



丸太
2年前期
内積や外積を用いた
衝突判定や
行列回転の実装



3Dシューティング
2年後期
オブジェクトプールや
相対座標を用いて追従させる
処理などを実装



3Dアクション
2年後期
回転する重力や
それに合わせてプレイヤーの
回転を調整する処理などを実装

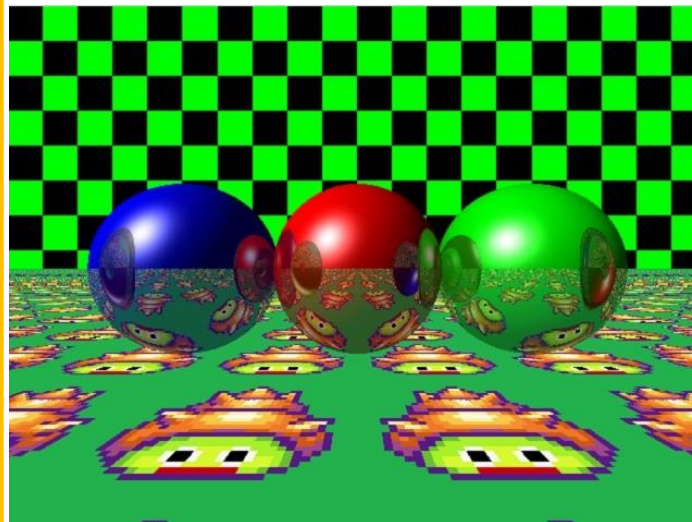
```
void Stage::ChangeStage(NAME type)
{
    activeName_ = type;
    // 対象のステージを取得する

    activePlanet_ = GetPlanet(activeName_);
    // ステージの当たり判定
    player_.ClearCollider(
    player_.AddCollider(ac
    //player_.lock()->Clea
    //player_.lock()->AddCollider(activePlanet_->GetTransform().collider);
    step_ = TIME_STAGE_CHANGE;
}
```

EX言語
2年後期
スマートポインタや関数ポインタ、
ラムダ式を学び、実装



頂点シェーダ
2年後期
初の頂点シェーダを使用して
ポストエフェクト等の実装



レイトレーシング
2年後期
モンテカルロ法などの
古典的レイトレーシングの実装



外部データ読込
2年後期
Unity上でモデルを配置し、
JSONで出力した
座標情報などを読込、
ゲーム上で表示する機能の実装